Department of Information Science & Engineering

Artificial Intelligence and Machine Learning

IS353IA

PROJECT SYNOPSIS Lab – Part B	
Title of Project:	Counterexample-Guided Auditing and Constrained LLM Repair of Learned Rewards for Protein Fitness Optimization
Student Name(s):	Varun Aditya, Ravi R
USN / Reg. No.:	1RV24IS143, 1RV25IS409
Semester and Section:	5th Semester, ISE B
Lab Batch:	B3
Date of Submission:	3rd October, 2026

Academic Year: 2026-27

1. Abstract

In protein engineering, lab experiments that measure how good a protein variant is are slow and costly. So machine learning (ML) models are trained to predict a variant's fitness, and a search algorithm uses the prediction as its reward to choose which variants to test next. The search keeps moving toward variants where the model is least reliable, so the predicted reward goes up but the measured fitness does not. Common fixes, such as a fixed uncertainty penalty or periodic retraining, use the same recipe no matter why the model failed. This project builds a framework that treats the reward as an explicit specification that can be checked and repaired. A large language model (LLM) converts a plain-language goal into a JSON specification made only of approved terms: predicted fitness, calibrated uncertainty, distance from labeled data, a mutation limit and a diversity term. A rule-based validator rejects anything else. An optimizer then picks variants in batches under a label budget, and their true labels are revealed. If the labels contradict the reward, the case is recorded as a counterexample of one of five types: miscalibration, support shift, model disagreement, constraint violation or diversity collapse. The framework applies the smallest suitable repair from a fixed list, checks it on already revealed labels, and then continues, rolls back or stops. Every change is logged. We will build it on frozen ESM-2 protein language model embeddings with an ensemble of regressors, and test it retrospectively on public experimental fitness datasets (FLIP, FLIP2 and ProteinGym) with labels hidden until queried. It will be compared with random, greedy, genetic and Bayesian optimization baselines, a fixed uncertainty penalty and retraining without diagnosis, using best fitness found, regret, calibration and recovery after failure.

Keywords: *Reward Repair, Proxy Reward Exploitation, Uncertainty Quantification, Conformal Prediction, Sequential Optimization, Protein Fitness Landscapes, Large Language Models*

2. Introduction

2.1 Background

Proteins are chains of amino acids. In protein engineering the aim is to find variants of a protein, with a few amino acids changed, that score higher on a measured property such as stability or binding. Each measurement needs a lab experiment, which is slow and

expensive, so ML models are trained to predict fitness from the sequence and to suggest which variants to test next.

Public benchmarks now let this be studied without a lab. ProteinGym has more than 250 standardized deep mutational scanning assays and supports both fitness prediction and design evaluation [1]. FLIP provides curated fitness landscapes whose splits test low-resource and extrapolative generalization [2]. FLIP2 adds seven datasets and 16 train-test splits that copy situations in real engineering campaigns, such as moving to more mutations, new positions or higher fitness [3]. A protein language model such as ESM-2 [4] can turn a sequence into a vector that a simple model can use as input.

The weak point is that a predictor is only reliable near the data it was trained on, while an optimizer deliberately moves away from that data. Optimizing a learned reward model too hard can reduce performance on the true objective, which is called reward overoptimization [5]. In biological sequence design, oracle models that differ only in architecture or training seed gave conflicting rankings of the same design methods, and poor out-of-distribution generalization was identified as a key cause [6]. Separately, LLMs have been used to write reward functions for reinforcement learning [7], [8], but those works were tested on robotics tasks and produce free-form code that is hard to verify.

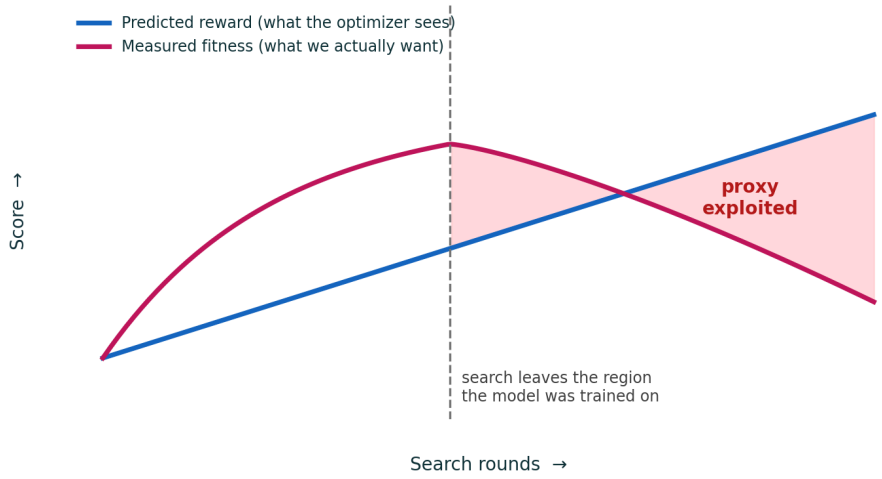


Figure 2.1: The problem in one picture. This is a schematic sketch to explain the idea, not experimental data.

Table 2.1: Terms used in this report

Term	Meaning in this report
Fitness	A measured number for a protein variant (for example stability or activity). Higher is better.
Surrogate / proxy reward	A trained ML model that predicts fitness so that not every variant has to be measured.
Reward specification	The recipe for the reward: which terms are used, their weights and the hard limits.
Counterexample	A queried variant whose true label contradicts what the current reward expected.
Conformal interval	A prediction interval built from past errors, aimed at containing the true value about 90% of the time.
Abstain	Stop querying and ask for more labels when no repair can be verified.

2.2 Problem Statement

When an optimizer with a fixed label budget maximizes a learned fitness predictor, it can exploit the predictor's errors outside its training distribution. The usual responses, a fixed uncertainty penalty or retraining, do not depend on why the failure happened. This project studies whether a contradiction between the reward and a revealed label can be recorded as a typed counterexample, used to pick a minimal and verified repair of an explicit reward specification, and judged by how well the search recovers afterwards.

2.3 Motivation

Lab budgets are limited, so a batch of measurements spent on variants that only looked good to the model is a real loss. The same effect also distorts how design methods are compared, since rankings can change with the oracle used [6]. FLIP2 now provides many shift-based splits on real measurements [3], so this failure can be studied on experimental data and not only on synthetic oracles. The people affected are protein engineering groups that use ML-guided campaigns, and ML researchers who compare sequence design methods.

3. Literature Survey / Review of Related Work

Sixteen references were used for this synopsis. They are discussed below under four themes: benchmarks, optimization with unreliable surrogates, reward overoptimization, and LLM-based reward design. Where we say a paper does not do something, this only refers to the paper as we read it. It is not a claim about all existing work.

3.1 Protein Fitness Benchmarks and Distribution Shift

Notin et al. [1] introduced ProteinGym, a set of more than 250 standardized deep mutational scanning assays for fitness prediction and design. Dallago et al. [2] introduced FLIP, which covers adeno-associated virus capsid viability, GB1 domain stability and binding, and thermostability, with splits that test low-resource and extrapolative generalization. Didi et al. [3] introduced FLIP2, with seven new datasets and 16 splits in five shift types (number, position, mutation, fitness and wild type). They report that wild-type, position and fitness splits are much harder than number and mutation splits, and that simple ridge regression on one-hot sequences often matches fine-tuned protein language models. This supports our plan to use simple models on frozen features and not fine-tune anything. Lin et al. [4] released the ESM-2 protein language models, with up to 15 billion parameters, whose learned representations can be used as sequence features. These resources give measured labels and realistic shifts, but they test predictors. They do not say what an optimizer should do when its reward starts failing during a search.

3.2 Optimization With Unreliable Surrogates

Trabucco et al. [9] proposed conservative objective models, which learn an objective that avoids overestimating values on out-of-distribution inputs and then optimize against it on a fixed dataset. Trabucco et al. [10] released Design-Bench, a benchmark suite for data-driven offline model-based optimization. Angermueller et al. [11] proposed DyNA-PPO, a reinforcement learning method for biological sequence design that trains the policy on a surrogate, picks the surrogate model from a pool in every round, and adds a count-based visitation bonus. Sinai et al. [12] proposed AdaLead, a simple adaptive greedy search that beat more complex methods on several biologically motivated tasks. Ren et al. [13] proposed Proximal Exploration, which prioritizes variants with few mutations. Bayesian optimization pairs a surrogate with an acquisition function [14], but the classic Gaussian-process version is best suited to continuous problems with fewer than 20 dimensions [14], while our inputs are discrete variants with embeddings of several hundred dimensions. We therefore take uncertainty from an ensemble [16] and wrap it with conformal prediction, which gives intervals with a chosen coverage level for any predictor [15]. In all of these methods, the response to unreliable predictions is fixed in advance: a penalty, a prior or a model-selection rule. None of them records a revealed failure as a typed event and picks a repair based on its cause.

3.3 Reward Overoptimization and LLM-Based Reward Design

Gao et al. [5] studied how optimizing a learned reward model affects a separate gold reward, and showed that pushing the proxy too hard can hurt true performance. They work on language-model alignment and describe the effect rather than repair it. Surana et al. [6] examined 12 sequence design methods and found that oracles differing in architecture, or only in training seed, gave conflicting relative performance. They propose adding biophysical measures to the evaluation and limiting out-of-distribution sequences the oracle must score. That is an evaluation-side remedy and does not change the reward during the search. Ma et al. [7] proposed Eureka, where an LLM writes reward code that is improved by evolutionary optimization, and reported that it beat human experts on 83% of 29 reinforcement learning environments. Xie et al. [8] proposed Text2Reward, which generates dense reward code from a natural-language goal and allows refinement with human feedback. Both generate free-form code for simulated robotics. Neither uses calibrated uncertainty, data-support checks or verified repair on a biological sequence objective.

3.4 Research Gap Summary

Table 3.1: Research gap summary

Reference	Key Contribution	Limitation / Research Gap
Notin et al. [1]; Dallago et al. [2]; Didi et al. [3]	Standardized fitness assays and shift-based splits	Test predictors, not reward failure and recovery inside a sequential search
Gao et al. [5]	Measured overoptimization of learned reward models (language-model alignment)	Describes the effect; not biological sequences; no repair mechanism
Surana et al. [6]	Showed that oracle choice changes method rankings across 12 sequence design methods	Evaluation-side advice; reward is not repaired during the search
Trabucco et al. [9]	Conservative objective model for offline optimization	One fixed remedy; static data; no revealed labels, diagnosis or audit
Angermueller et al. [11]	Model-based RL with per-round surrogate selection	Surrogate chosen by model fit; no typed failure diagnosis, rollback or abstention
Sinai et al. [12]; Ren et al. [13]	Strong search heuristics for sequence design	Fixed heuristics; no monitoring of reward reliability
Ma et al. [7]; Xie et al. [8]	LLM-generated reward code for RL	Free-form code in robotics; no uncertainty-based verification; not a biological setting
Angelopoulos and Bates [15]	Distribution-free prediction intervals	Gives a reliability signal, not a policy for repairing the reward

Gap addressed by this project. In the papers we reviewed, we did not find a protocol that (i) records a contradiction between a reward and a revealed label as a typed counterexample, (ii) picks the smallest allowed change to an explicit reward specification based on the diagnosed cause, (iii) verifies the change before using it, with rollback and abstention, and (iv) reports recovery after failure on experimental fitness data.

4. Objectives

The primary objectives of this project are:

- To design a typed reward-specification schema with a rule-based validator, and to implement a constrained LLM interface that turns a plain-language goal into that schema using only approved terms.
- To implement a budgeted sequential search pipeline using frozen ESM-2 embeddings and an ensemble surrogate with conformal calibration, together with detectors for five counterexample types: miscalibration, support shift, model disagreement, constraint violation and diversity collapse.
- To develop a fixed repair library with verification, rollback and abstention, plus a versioned audit log, and to define a repair cost so that alternative repairs can be compared.
- To evaluate the framework on public experimental datasets (FLIP, FLIP2, ProteinGym) under equal label budgets against random, greedy, genetic, AdaLead-style, Bayesian optimization, fixed-penalty and retraining baselines, with ablations, and to report best fitness found, regret, calibration and recovery after failure.

5. Proposed Methodology

5.1 System Architecture Overview

The framework is a loop with four stages: specify, optimize, falsify and repair (Figure 5.1). In the specify stage, a frozen LLM turns the user's goal into a JSON reward specification, and a rule-based validator accepts it only if every term is on an approved list and every weight is inside its allowed range. In the optimize stage, an ensemble trained on frozen ESM-2 embeddings predicts fitness and uncertainty, and a batch of variants is chosen from a finite pool of variants that have already been measured in a

public dataset. In the falsify stage, the true labels of that batch are revealed and five detectors compare them with what the reward expected. Each failed check becomes a typed counterexample. In the repair stage, the system maps the failure type to candidate repairs from a fixed library and applies the cheapest one that passes verification. If none passes, it keeps the old reward, and it stops after repeated failures. Every reward version, trigger and repair is written to an audit log.

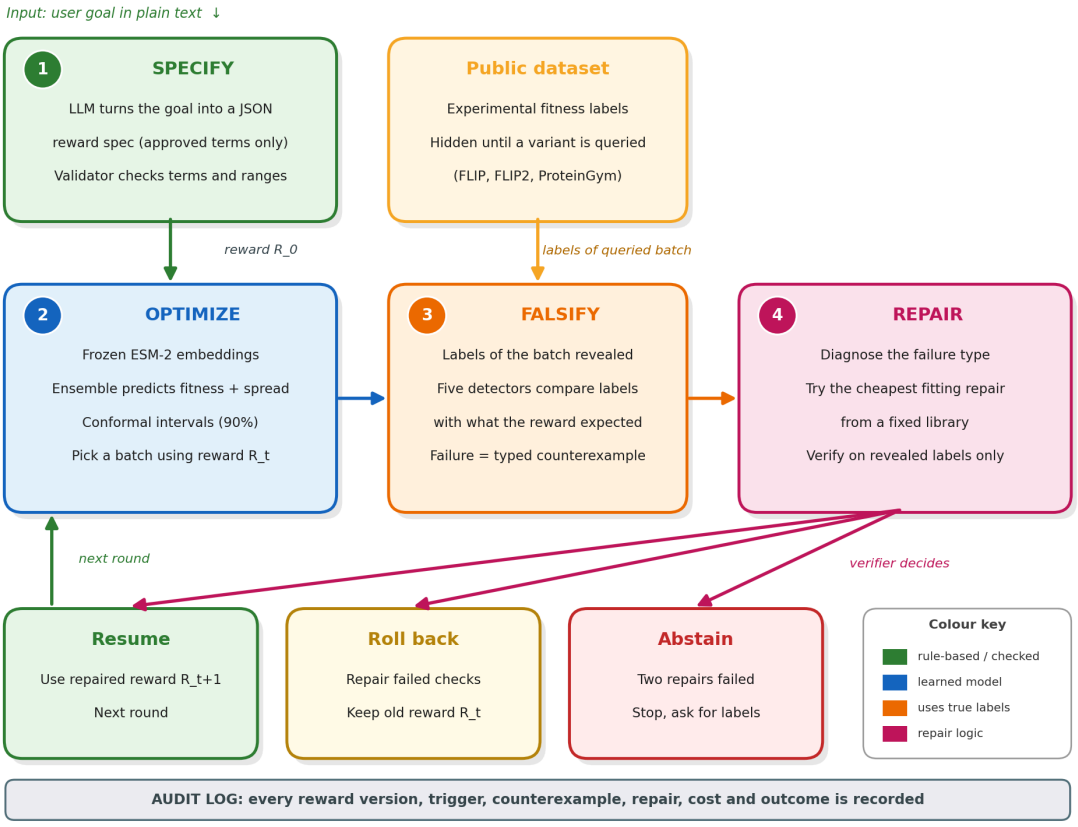


Figure 5.1: Proposed system architecture

What the LLM does and does not do. The LLM has two jobs. It compiles the goal into the specification, and, when a counterexample is recorded, it picks one repair from the library and a parameter value inside the allowed range. It never scores a variant, never writes reward code and never sees labels that have not been revealed. Its output is checked by the validator, retried once if invalid, and replaced by a rule-based default if it is invalid again. The format of a specification is shown below. The weights are placeholders to show the format.

```
{
  "goal": "maximize_fitness",
  "terms": {
    "uncertainty_penalty": {"weight": 0.5, "allowed_range": [0.0,
2.0]},
    "distance_penalty":    {"weight": 0.3, "allowed_range": [0.0,
2.0]},
    "diversity_bonus":     {"weight": 0.1, "allowed_range": [0.0,
1.0]}
  },
  "hard_constraints": {"max_mutations": 4},
  "budget": {"rounds": 10, "batch_size": 10},
  "abstain_after_failed_repairs": 2
}
```

5.2 Dataset Description

All experiments use public datasets of experimentally measured fitness. The optimizer never sees a label until it queries that variant, so a run simulates a lab campaign. The FLIP2 files are CSVs with the columns sequence, target, set and validation [3]. Table 5.1 lists the planned landscapes. The first three are chosen now because their sizes are small enough for our compute and their splits create the shifts we want to study. The last two are optional.

Table 5.1: Planned datasets

Landscape	Source and size	Split used to build the seed set	Planned role
Alpha amylase	FLIP2 [3]; 3,706 variants (488 single mutants), up to 8 mutations; stain-removal activity; MIT license	close-to-far (position shift)	Development (tune thresholds)
Imine reductase (IRED)	FLIP2 [3]; 17,143 variants, up to 15 mutations; activity from a microfluidic screen; CC-BY 4.0	two-to-many (number shift)	Test
Hydrophobic core	FLIP2 [3]; 24,935 variants of three wild-type proteins; stability; CC-BY 4.0	low-to-high (fitness shift)	Test
GB1	Original FLIP [2]; domain stability and binding; size to be recorded in Phase II	low_vs_high	Optional fourth landscape
ProteinGym assay	ProteinGym [1], proteingym.org; one assay chosen in Phase II	to be chosen	Optional extra check

Together the first three landscapes have 45,784 sequences to embed. We leave out two FLIP2 datasets on purpose: Nuclease B stores its target as four activity bins and not a continuous value, and TrpB has 228,298 variants, which is too large for our compute [3].

Pre-processing. Fitness is a continuous number, so there are no classes to balance. We will check the fitness distribution of each landscape during exploratory data analysis. Planned steps: remove duplicate sequences and sequences with non-standard residues, embed every sequence once with ESM-2 by averaging over residues, and cache the embeddings. We also compute each variant's mutations and its Hamming distance from the wild type. For reporting only, fitness is rescaled with the landscape's own minimum and maximum. Those two numbers are never given to the optimizer.

Search protocol. The candidate pool is every measured variant in the landscape. The seed set is drawn from the training side of the chosen split, for example only low-fitness variants in Hydro, so that the search has to move into the shifted region where failures are expected. Planned starting values are 100 seed labels, 10 rounds and a batch of 10 per round. Every method gets the same seed set and the same budget, and each setting is run with 10 random seeds.

5.3 Proposed Algorithm / Model

Surrogate. The surrogate is an ensemble of 5 to 10 regressors on the ESM-2 embeddings of labeled variants. We plan to start with the 8M-parameter ESM-2 model, which gives 320-dimensional vectors, and move to a larger one only if time allows. Members are ridge regression (regularization chosen by cross-validation) and a small multilayer perceptron with two hidden layers (256 and 128 units, ReLU), mean-squared-error loss, Adam (learning rate 1e-3, weight decay 1e-4) and early stopping. Each member sees a bootstrap sample of the labeled set. The ensemble mean is the predicted fitness $\mu(x)$ and the ensemble standard deviation is the uncertainty $\sigma(x)$ [16]. Split-conformal prediction on a held-out 20% of the revealed labels turns this into an interval with a 90% target [15]. The distance term $d(x)$ is the distance from x to its nearest labeled variant in embedding space, scaled to [0, 1] using the labeled set.

One limit has to be stated. The coverage guarantee of conformal prediction assumes the calibration data and the new points are exchangeable [15]. A search that moves toward high predicted reward breaks this on purpose. So we do not assume the 90% holds during the search. We measure the real coverage on each revealed batch, and a drop in coverage is one of the signals the framework reacts to.

Reward. In round t the reward is built from approved terms:

$$R_t(x) = \mu_t(x) - \lambda_t \cdot \sigma_t(x) - \rho_t \cdot d_t(x) - \eta \cdot \Sigma_j \max(0, g_j(x)) + \beta_t \cdot D(x, B_t)$$

Here $g_j(x) \leq 0$ are the hard constraints (for example a maximum number of mutations), $D(x, B_t)$ is how far x is from the variants already chosen for the current batch, and λ, ρ, η and β are weights with allowed ranges stored in the specification. This formula is not our contribution. Uncertainty penalties, conservative objectives and diversity terms already exist [9], [11]. A batch is built greedily: each next variant is the one with the highest R_t given the variants already picked.

Counterexample detectors. Table 5.2 lists the five detectors, their planned trigger rules and the repairs they allow. All numbers are starting values and will be tuned only on the development landscape. The miscalibration rule uses the last 30 revealed labels (three batches) because a single batch of 10 is too noisy. If the true coverage is 0.90, coverage measured on 30 labels has a standard deviation of about 0.055, so the 0.75 trigger sits about 2.7 standard deviations below it.

Table 5.2: Counterexample types, triggers and repairs

Type	Trigger (planned starting value)	Allowed repairs
Constraint violation	A selected variant breaks a hard constraint $g_j(x) \leq 0$. Constraints are applied as a filter before selection, so this mainly tests the filter and any edited specification.	Reject the variant, fix the filter, keep the old reward
Support shift	At least 3 variants in the batch are outside their conformal interval and have $d(x)$ above the 90th percentile of the labeled set.	Shrink the trust radius (maximum distance from labeled variants); raise ρ
Miscalibration	Coverage on the last 30 revealed labels is below 0.75 (target 0.90).	Refit the conformal quantile and retrain the ensemble; raise λ
Model disagreement	Mean pairwise Spearman correlation between ensemble members on the top 100 candidates is below 0.5.	Use the median or a lower quantile of the members; raise λ
Diversity collapse	Mean pairwise Hamming distance in the batch is below 2.	Raise β or select the batch by farthest-point selection

If several detectors fire in the same round, they are handled one at a time in this order: constraint violation, support shift, miscalibration, model disagreement, diversity collapse, with a re-check after each repair. Conflict between several measured properties is out of scope, because each chosen dataset reports a single property.

Repair cost and verification. Let θ be the vector of reward parameters.
The cost of changing reward R into R′ is -
$C(R', R) = \Sigma_j |\theta'_j - \theta_j| / \text{range}_j + c \cdot (\text{number of terms changed})$.

For a diagnosed failure the system tries the allowed repairs from cheapest to most expensive and keeps the first one that passes four checks, all computed from labels that are already revealed: (1) if the round were replayed under R′, the counterexample variants would not be picked again; (2) all hard constraints still hold; (3) coverage on the calibration split is within a set tolerance of the target; (4) rank correlation on earlier labeled variants does not drop by more than a set margin. If no repair passes, the old reward is kept, and after two failed repairs in a row the system abstains.

Worked example. The numbers here are made up to explain the steps and are not results. After round 5, 21 of the last 30 revealed labels fall inside the 90% interval, so coverage is 0.70, below the 0.75 trigger. In the newest batch, 3 variants are outside their interval and all three have a distance d(x) above the 90th percentile. Support shift and miscalibration both fire. Support shift is handled first. Shrinking the trust radius from 4 to 3 mutations is the cheapest allowed repair, and it passes the four checks, so it becomes the new reward. After the next batch is revealed, the miscalibration detector is checked again. If coverage is still low, the conformal quantile is refit. Each step is written to the audit log.

Algorithm (numbered steps).

1. Compile the user's goal into a reward specification with the LLM and validate it. Stop if it is rejected.
2. Embed all variants once, draw the seed set and reveal its labels.
3. Train the ensemble and calibrate the conformal intervals.
4. Score the pool with R_t and pick a batch of B variants greedily with the diversity term.
5. Reveal the labels of the batch and add them to the labeled set.
6. Run the five detectors. If none fires, retrain and continue with R_t.
7. For each fired detector, store a counterexample record with its type and evidence.
8. Take the allowed repairs for that type and check them in order of cost.
9. Accept the first repair that passes verification as R_(t+1). Otherwise keep R_t. Abstain after two failed repairs in a row.
10. Append version, trigger, repair, cost and outcome to the audit log, and repeat until the label budget is used up.

Search methods and baselines. The default optimizer is the greedy batch selection above. The genetic-algorithm and AdaLead-style baselines [12] only propose variants

from the measured pool that are within a Hamming distance of 2 of the current best-ranked variants, so that every proposal has a label to reveal. The Bayesian optimization baseline [14] uses ensemble mean and standard deviation for upper-confidence-bound and expected-improvement acquisition on the discrete pool, since a Gaussian process is a poor fit for embeddings with hundreds of dimensions. Reinforcement learning is an optional extra baseline: a PPO policy that builds a variant through mutation steps inside the surrogate and receives R_t as its reward, in the style of [11]. It is added only after the main pipeline works, and its result will be reported whether or not it wins.

Novel versus adopted. Adopted from earlier work: ESM-2 embeddings [4], ensembles [16], conformal prediction [15], uncertainty-penalized and diversity-aware rewards [9], [11], and Bayesian optimization, genetic and AdaLead-style search [12], [14]. Proposed in this project: the typed counterexample record for a reward specification, the repair library with a repair cost, the verification, rollback and abstention rules, the audit log, and the recovery-focused evaluation. We will also test whether the LLM adds anything by replacing it with a rule-based selector in an ablation.

5.4 Implementation Plan (Phase-wise)

Table 5.3: Phase-wise implementation plan

Phase	Task to be completed	Activities involved	Duration
I	Literature Review & Problem Definition	Read the key papers; finalize the problem statement, the five counterexample types and the scope	2 weeks (Weeks 1–2)
II	Dataset Acquisition & Pre-processing	Download Amylase, IRED and Hydro (and GB1 or a ProteinGym assay if used); clean sequences; exploratory analysis; compute and cache ESM-2 embeddings	2 weeks (Weeks 3–4)
III	System Design & Architecture	Define the reward schema and validator; design detectors, repair library and audit-log format; set up the environment and repository	2 weeks (Weeks 5–6)
IV	Implementation & Training	Build the ensemble and conformal calibration; implement baselines, detectors, repairs, verifier and LLM interface; tune thresholds on Amylase only	5 weeks (Weeks 7–11)
V	Evaluation & Testing	Run all methods with 10 seeds on the shift splits and injected failures; ablations; statistical tests; optional PPO baseline	3 weeks (Weeks 12–14)
VI	Documentation & Reporting	Final report, paper draft, presentation	4 weeks (Weeks 15–18)

5.5 Tools & Technologies

Table 5.4: Tools and technologies

Category	Tools / Technologies / Versions
Programming Language	Python 3.10 or later
Frameworks / Libraries	PyTorch; Hugging Face Transformers (ESM-2 checkpoints); scikit-learn; NumPy; pandas; SciPy; Pydantic (to validate the reward specification); matplotlib; optional: MAPIE, Stable-Baselines3 (PPO baseline)
Development Environment	Google Colab and VS Code with Jupyter Notebook
Database / Storage	Parquet or HDF5 for cached embeddings; JSON Lines and SQLite for the audit log
Hardware / Platform	Google Colab T4 GPU for the one-time embedding step; laptop with 16 GB RAM for the optimization loop
Version Control	Git / GitHub
Other Tools	An instruction-following LLM (gpt-oss-120b) used through an API (Groq Cloud) with JSON output at temperature 0; Weights & Biases (optional) for experiment tracking

5.6 Risks and Mitigation

Table 5.5: Main risks and how they are handled

Risk	Why it matters	Mitigation
Detectors are noisy with batches of 10	A false alarm triggers an unnecessary repair	Use the last 30 labels for coverage; tune thresholds on the development landscape; report false-alarm rate
Conformal coverage is not guaranteed during the search	Exchangeability is broken by the search itself	Measure real coverage per batch and treat a drop as a signal; report it honestly
The LLM adds nothing or returns invalid JSON	The LLM part would be decoration	Validator, one retry, rule-based fallback, and an ablation without the LLM
Large landscapes are too slow	Embedding and repeated retraining cost compute	Use only Amylase, IRED and Hydro; cache embeddings; start with the 8M ESM-2 model
Plain Bayesian optimization matches our method	No measurable gain on fitness	Report it as a negative result; the contribution is then the failure analysis and the protocol
Assay noise looks like a failure	A noisy label can be mistaken for a counterexample	Use interval-based rules and several labels, not one residual
Thresholds overfit one dataset	Results would not generalize	Tune on Amylase only and report on the other landscapes

6. Expected Outcomes & Deliverables

Upon successful completion of this project, the following outcomes are expected:

6.1 Technical Outcomes

- A reproducible code repository with the embedding cache, ensemble surrogate with conformal calibration, optimizers, detectors, repair library, verifier and audit log.
- Results on three landscapes (Amylase for development, IRED and Hydro for testing), at least two shift types, 10 seeds each, and the seven baselines plus ablations listed in Section 7, all under the same label budget.
- Conformal interval coverage after repair within ±5 percentage points of the 90% target on the test landscapes. This is a target to be tested.
- At least 10% lower post-failure simple regret than the fixed-uncertainty-penalty baseline on the shift splits. This is also a target and not an assumed result. If it is not reached, we will report that.
- A table of precision and recall for each detector on deliberately injected failures of known type.

6.2 Research Outcomes

- A short paper draft if the results are good enough to write up, aimed at a machine learning for biology workshop or similar venue.
- A clearly defined counterexample-guided reward-repair procedure and recovery-focused evaluation, with measured comparisons against the baselines.

6.3 Deliverables Checklist

Table 6.1: Deliverables checklist

Deliverable	Target Timeline
Synopsis / Proposal Document	Week 2
Literature Review Report	Week 3
Working Prototype / Baseline Model	Week 8
Final Trained Model + Code Repository	Week 14
Evaluation Report & Results	Week 14
Journal / Conference Paper Draft	Week 17
Final Project Report & Presentation	Week 18

7. Evaluation Metrics & Validation Plan

All metrics are computed against the hidden experimental labels, never against the reward the optimizer used.

- **Optimization quality:** best-so-far true fitness (rescaled), simple regret (best fitness in the landscape minus best found), cumulative regret, and recall of the top 1% of variants.
- **Calibration:** observed interval coverage against the 90% target, mean interval width, and Spearman correlation between predicted and true fitness on each queried batch.
- **Failure detection:** precision, recall and F1 of each detector on injected failures of known type, and the false-alarm rate on runs without injected failures.
- **Recovery and repair behavior:** rounds to recovery (rounds after the first repair until coverage on the newest batch is back within 0.05 of the target and the batch's mean true fitness is no lower than before the failure), number of repairs, repair cost C, rollback rate, and abstention rate.
- **Constraints and diversity:** constraint-violation rate and mean pairwise Hamming distance of selected batches.
- **System cost:** number of LLM calls and run time per experiment.

Validation Strategy:

- Retrospective protocol: labels hidden until queried; same seed set, batch size and budget for every method; 10 random seeds per setting, reported as the mean with 95% bootstrap confidence intervals; paired Wilcoxon signed-rank tests across seeds.
- Shift scenarios: the FLIP2 splits in Table 5.1 (position shift on Amylase, number shift on IRED, fitness shift on Hydro). We also inject failures with a known cause: a seed set limited to variants with at most two mutations (support shift), a very small calibration set (miscalibration), a disabled mutation-count filter (constraint violation), and no diversity term (diversity collapse).
- Thresholds and hyperparameters are tuned on Amylase and with cross-validation on revealed labels only. The test landscapes are not used for tuning.
- Baselines: random search, greedy predicted-fitness selection, genetic algorithm, AdaLead-style search, Bayesian optimization, fixed uncertainty penalty, and periodic retraining without diagnosis. Ablations: no LLM (rule-based repair selector), no verifier, no rollback, no abstention, one universal penalty instead of type-specific repairs, and each repair removed in turn. Optional: PPO.

Ethical Considerations:

- Only public datasets are used, with citation. The FLIP2 datasets we plan to use are released under CC-BY 4.0, except Amylase, which is under the MIT license [3]. No human subjects, patient data or wet-lab work is involved, so ethics approval is not needed.
- The study is retrospective and only selects among variants that were already measured. It does not generate or release new sequences, and it makes no claim that any variant is functional or safe.
- Results will be reported for each landscape and shift type, and not only as averages, so that poor results on one dataset are not hidden. Negative results will be reported.
- The LLM only receives the user's goal text and summary statistics of revealed batches. Prompts, outputs and random seeds are logged so that runs can be reproduced.

8. References

[1] P. Notin et al., "ProteinGym: Large-scale benchmarks for protein fitness prediction and design," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, vol. 36, 2023.

[2] C. Dallago et al., "FLIP: Benchmark tasks in fitness landscape inference for proteins," in Proc. NeurIPS Datasets and Benchmarks Track, 2021.

[3] K. Didi et al., "FLIP2: Expanding protein fitness landscape benchmarks for real-world machine learning applications," in Proc. 43rd Int. Conf. Machine Learning (ICML), 2026. bioRxiv doi: 10.64898/2026.02.23.707496. [Online]. Available: https://flip.protein.properties

[4] Z. Lin et al., "Evolutionary-scale prediction of atomic-level protein structure with a language model," Science, vol. 379, no. 6637, pp. 1123–1130, 2023. doi: 10.1126/science.ade2574.

[5] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," in Proc. Int. Conf. Machine Learning (ICML), 2023. arXiv:2210.10760.

[6] S. Surana, N. Grinsztajn, T. Atkinson, P. Duckworth, and T. D. Barrett, "Overconfident oracles: Limitations of in silico sequence design benchmarking," arXiv:2502.17246, 2025 (ICML 2024 AI for Science Workshop).

[7] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2024. arXiv:2310.12931.

[8] T. Xie et al., "Text2Reward: Reward shaping with language models for reinforcement learning," in Proc. ICLR, 2024. arXiv:2309.11489.

[9] B. Trabucco, A. Kumar, X. Geng, and S. Levine, "Conservative objective models for effective offline model-based optimization," in Proc. ICML, PMLR vol. 139, 2021. arXiv:2107.06882.

[10] B. Trabucco, X. Geng, A. Kumar, and S. Levine, "Design-Bench: Benchmarks for data-driven offline model-based optimization," in Proc. ICML, PMLR vol. 162, 2022. arXiv:2202.08450.

[11] C. Angermueller et al., "Model-based reinforcement learning for biological sequence design," in Proc. ICLR, 2020.

[12] S. Sinai, R. Wang, A. Whatley, S. Slocum, E. Locane, and E. D. Kelsic, "AdaLead: A simple and robust adaptive greedy search algorithm for sequence design," arXiv:2010.02141, 2020.

[13] Z. Ren, J. Li, F. Ding, Y. Zhou, J. Ma, and J. Peng, "Proximal exploration for model-guided protein sequence design," in Proc. ICML, PMLR vol. 162, 2022.

[14] P. I. Frazier, "A tutorial on Bayesian optimization," arXiv:1807.02811, 2018.

[15] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," Found. Trends Mach. Learn., vol. 16, no. 4, 2023. doi: 10.1561/2200000101.

[16] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017. arXiv:1612.01474.

9. Declaration & Signatures

We hereby declare that this project synopsis is our original work carried out as part of the AI and ML course. This work has not been submitted elsewhere for any other degree or examination. All sources of information have been duly cited and acknowledged.

Student Signature:

Varun Aditya (1RV24IS143)

Ravi R (1RV25IS409)

Date: ____

Guide Signature:

Dr. Merin Meleet (Associate Professor, Department of ISE)

Date: ____